

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS



CRIPTOGRAFÍA Y SEGURIDAD

TAREA 03

Alumno:

■ Paredes Zamudio Luis Daniel 318159926

Profesor: José Antonio Macías García

18 de Mayo de 2026

Ejercicio 4: Una clave y un secreto en AES

La lógica del script `solution.py` es la siguiente:

- Cargamos los archivos `file.txt`, `hashes.txt` y `cipher.txt` y limpiamos su contenido (quitamos acentos, pasamos a minúsculas...) para trabajar únicamente con un alfabeto de 26 letras sin ruido.
- Generamos los primeros 50 números primos con `sympy` y los usamos como guía para armar cadenas de 50 letras de longitud. Para cada primo p se toma una letra en una posición determinada del texto y, en algunas variantes, además le aplica un desplazamiento César dependiente de $p-2$.
- A c/u de estas candidatas se le prueba con algunas combinaciones con la palabra base `kevinmitnick`, mezclando usando Vigenère por suma/resta y una intercalación de caracteres.
- Para cada una de estas combinaciones se calcula su SHA-256 y se compara con los hashes dados en `hashes.txt`. Cuando alguno coincide se derivan las claves necesarias para descryptar `cipher.txt` con AES-GCM (nonce de 12 bytes);
- Si la autenticación pasa, se imprime el mensaje en la pantalla.

El código completo del script se encuentra a continuación:

```
1
2 import hashlib
3 import unicodedata
4
5 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
6 from sympy import primerange
7
8
9 def ejercicio_cuatro() -> None:
10
11     try:
12         with open("file.txt", "r", encoding="utf-8") as f:
13             texto = f.read()
14         with open("hashes.txt", "r", encoding="utf-8") as f:
15             # Cargamos los hashes validos en un set para busqueda eficiente.
16             # Se asume que cada linea del archivo contiene un hash SHA-256 valido.
17             hashes_validos = set(line.strip() for line in f if line.strip())
18             # Cargamos los datos cifrados en formato hexadecimal, eliminando cualquier espacio o
19             # salto de linea.
20             with open("cipher.txt", "r", encoding="utf-8") as f:
21                 cipher_hex = f.read().replace("\n", "").strip()
22     except FileNotFoundError as e:
23         print(f"[-] Error: Archivo no encontrado - {e}")
```

```

23     return
24
25     # Se eliminan acentos, simbolos y saltos de linea.
26     # Conservamos unicamente letras minusculas (a-z) para asegurar un bloque continuo.
27     texto_norm = (unicodedata.normalize("NFKD", texto).encode("ascii", "ignore").decode("utf
-8"))
28     texto_filtrado = "".join(c for c in texto_norm if c.isalpha()).lower()
29
30     # Generamos los primeros 50 numeros primos utilizando sympy.primerange
31     primos = [p for _, p in zip(range(50), primerange(2, 1000))]
32     palabra_base = "kevinmitnick"
33
34     # El uso de '% len(texto_filtrado)' garantiza que el texto funcione como un "loop
infinito".
35     # por que si el indice calculado excede la longitud del texto, simplemente vuelve al
inicio.
36     opciones_extraccion = [
37         # A: El indice extraido es el numero primo restandole 2.
38         "".join(texto_filtrado[(p - 2) % len(texto_filtrado)] for p in primos),
39         # B: Se usa el numero primo como indice, y se aplica un desplazamiento Cesar de (p
-2).
40         "".join(
41             chr((ord(texto_filtrado[p % len(texto_filtrado)]) - 97 + (p - 2)) % 26 + 97)
42             for p in primos
43         ),
44         # C: Indice en (p-2) y desplazamiento Cesar en (p-2).
45         "".join(chr((ord(texto_filtrado[(p - 2) % len(texto_filtrado)]) - 97 + (p - 2)) %
26+ 97)
46             for p in primos
47         ),
48     ]
49
50     clave_correcta = None
51     hash_correcto = None
52
53     for extraido in opciones_extraccion:
54         # Exploramos las diferentes formas de "mezclar" el texto con la palabra base "
kevinmitnick"
55         mezclas = [
56             extraido,
57             # Mezcla por suma (Estilo Cifrado Vigenere). Por ejemplo, si extraido = "abcde
..." y
58             # palabra_base = "kevinmitnick", la mezcla seria: "kkbcvindemnitck..."
59             # (alternando caracteres de ambos strings pero sumando sus valores).
60             "".join(chr((ord(extraido[i]) - 97+ ord(palabra_base[i % len(palabra_base)]) - 97)
% 26+ 97)

```

```

61         for i in range(50)),
62         # Mezcla por resta. Por ejemplo, si extraido = "abcde..." y palabra_base = "
        kevinmitnick", la mezcla seria:
63         # "akbcvindemnitck..." (alternando caracteres de ambos strings pero restando sus
        valores).
64         "".join(chr((ord(extraido[i]) - ord(palabra_base[i % len(palabra_base)])) % 26+
        97)

65         for i in range(50)),
66         # Se toma el caracter i-esimo de 'extraido' y se concatena con el caracter i-
        esimo de 'palabra_base' (ciclico).
67         # Por ejemplo, si extraido = "abcde..." y palabra_base = "kevinmitnick", la
        mezcla seria:
68         # "akebcvindemnitck..." (alternando caracteres de ambos strings)
69         "".join(extraido[i] + palabra_base[i % len(palabra_base)] for i in range(25)),
70     ]
71
72     # Comparamos el SHA-256 de cada combinacion generada contra la lista de control
73     for candidata in mezclas:
74         h = hashlib.sha256(candidata.encode()).hexdigest()
75         if h in hashes_validos:
76             clave_correcta = candidata
77             hash_correcto = h
78             break
79     if clave_correcta:
80         break
81
82     if not clave_correcta:
83         print("[-] No se encontro ninguna clave que coincida con los hashes.")
84         return
85
86     print(f"\n[+] Clave confirmada: {clave_correcta}")
87     print(f"[+] Hash coincidente : {hash_correcto}")
88
89     try:
90         data = bytes.fromhex(cipher_hex)
91     except ValueError:
92         print("[-] Error: cipher.txt no contiene un formato hexadecimal valido.")
93         return
94
95     nonce = data[:12] # AES-GCM estandar usa un Nonce (IV) de 12 bytes
96     ciphertext = data[12:] # El resto del bloque es el texto cifrado + Tag de autenticacion
97
98     llaves_a_probar = {
99         "Hash Directo a Bytes (Lectura Literal)": bytes.fromhex(hash_correcto),
100        "Doble Hash (Lectura sugerida por Hint)": hashlib.sha256(hash_correcto.encode()).
        digest(),

```

```
101     "Derivacion Original (Hash de Clave Extr)": hashlib.sha256(clave_correcta.encode()).
    digest(),
102 }
103
104 exito = False
105 for _, key_bytes in llaves_a_probar.items():
106     aesgcm = AESGCM(key_bytes)
107     try:
108         # Si decrypt() no lanza la excepcion 'InvalidTag', significa que la llave derivo
    exitosamente.
109         decrypted = aesgcm.decrypt(nonce, ciphertext, None)
110         print("[+] Mensaje Descifrado:\n")
111         print(f"{decrypted.decode('utf-8')}")
112         exito = True
113         break
114         # La llave es incorrecta para este ciphertext. Ignoramos y probamos la siguiente
    .
115     except Exception:
116         continue
117
118 if not exito:
119     print("[-] Error: Ninguna de las llaves derivadas logro descifrar el mensaje.")
120
121 if __name__ == "__main__":
122     ejercicio_cuatro()
```

Y el mensaje desencriptado es el siguiente:

Mitnick gained unauthorized access to a computer network in 1979, at 16, when a friend gave him the telephone number for the Ark, the computer system that Digital Equipment Corporation (DEC) used for developing its RSTS/E operating system software. He broke into DEC's computer network and copied the company's software, a crime for which he was charged and convicted in 1988. He was sentenced to 12 months in prison followed by three years of supervised release. Near the end of his supervised release, Mitnick hacked into Pacific Bell voicemail computers. After a warrant was issued for his arrest, Mitnick fled, becoming a fugitive for two-and-a-half years.

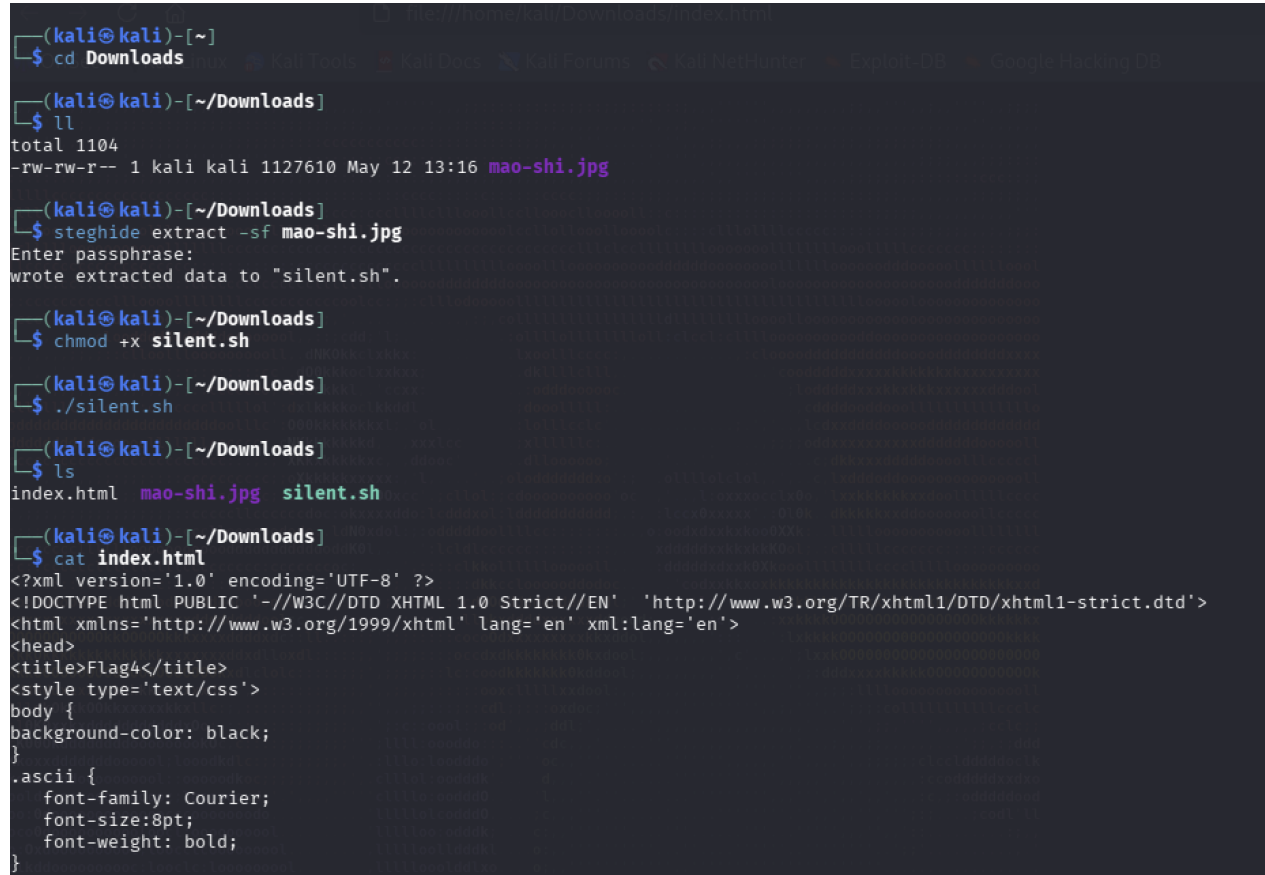
Ejercicio 5: El gran secreto dentro de una imagen

Info. VM 1

OS: Kali GNU/Linux Rolling x86_64 Kernel version: 6.19.1+kali-amd64 Núcleos: 2 x Intel Core i7-9750H RAM: 2GB IP: 192.168.1.230

Para este ejercicio se utilizó la imagen `mao-shi.jpg` indicada en el repositorio de la tarea y se ocuparon los mismos comandos mencionados en el mismo.

Primero, tras la instalación de `steghide` y `jp2a`, se ejecutó el comando `steghide extract -sf mao-shi.jpg`, obteniendo lo siguiente:



```
(kali@kali)-[~]
$ cd Downloads
(kali@kali)-[~/Downloads]
$ ll
total 1104
-rw-rw-r-- 1 kali kali 1127610 May 12 13:16 mao-shi.jpg
(kali@kali)-[~/Downloads]
$ steghide extract -sf mao-shi.jpg
Enter passphrase:
wrote extracted data to "silent.sh".
(kali@kali)-[~/Downloads]
$ chmod +x silent.sh
(kali@kali)-[~/Downloads]
$ ./silent.sh
(kali@kali)-[~/Downloads]
$ ls
index.html  mao-shi.jpg  silent.sh
(kali@kali)-[~/Downloads]
$ cat index.html
<?xml version='1.0' encoding='UTF-8' ?>
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN" 'http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd'>
<html xmlns='http://www.w3.org/1999/xhtml' lang='en' xml:lang='en'>
<head>
<title>Flag4</title>
<style type='text/css'>
body {
background-color: black;
}
.ascii {
font-family: Courier;
font-size:8pt;
font-weight: bold;
}
```

Figura 1: Obtención de archivos ocultos en `mao-shi.jpg`

Luego, aunque el archivo `silent.sh` genera un archivo `html` con el que podemos continuar la búsqueda de la bandera escondida, se optó por simplemente hacer `cat silent.sh` para visualizar en pantalla el contenido del archivo, encontrando al final del archivo la bandera `FLAG{s73g4n0graphy_15_hidd3n}`

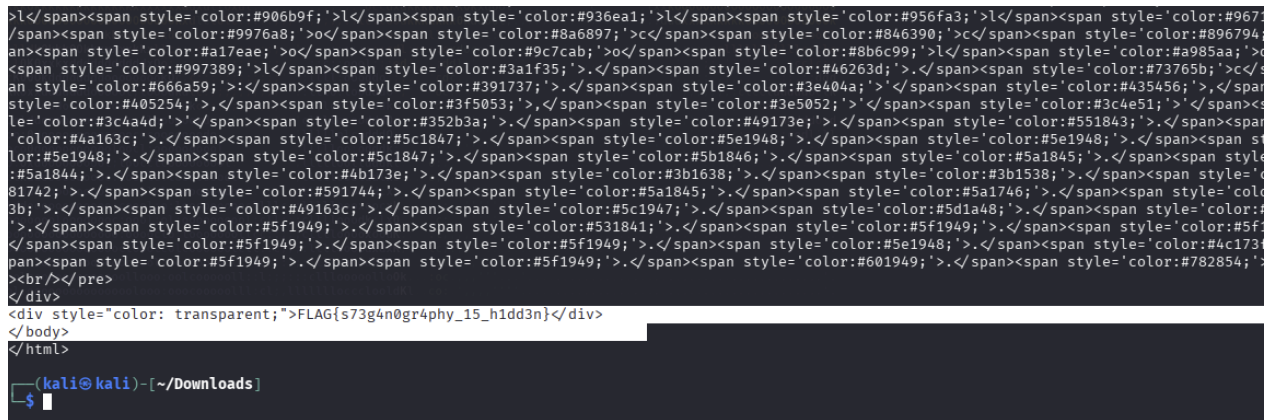


Figura 2: Obtención de la bandera escondida en index.html

Ejercicio 6: Una gran tormenta se avecina

Ejercicio 7: ¡Hey! No mires más de lo necesario

Info. VM 2

OS: Debian GNU/Linux 13 (trixie) Kernel version: 6.12.86+deb13-amd64 Núcleos: 2 x Intel Core i7-9750H RAM: 2GB IP: 192.168.1.231

Se hacen las configuraciones mencionadas en la descripción de la tarea para la configuración de la VM Debian.

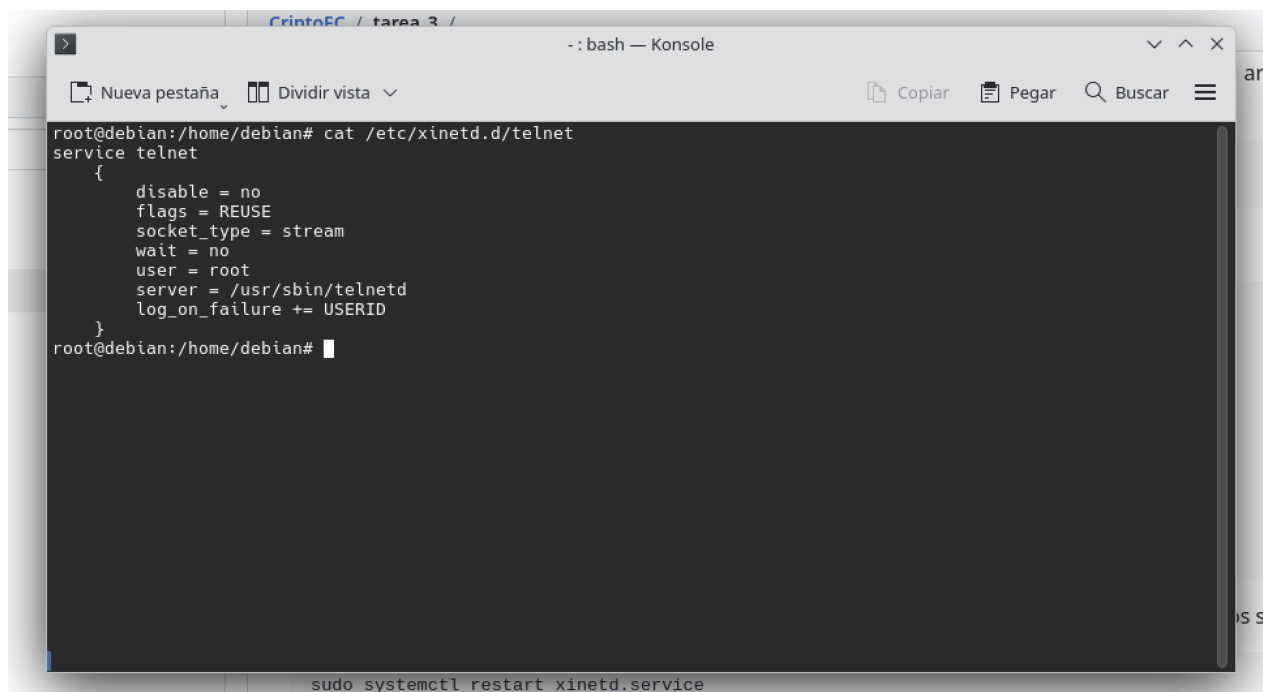


Figura 3: Configuración de telnet en la VM Debian

Luego, ponemos el fitro indicado en Wireshark y ejecutamos `telnet 192.168.1.231`. Estos son los resultados:

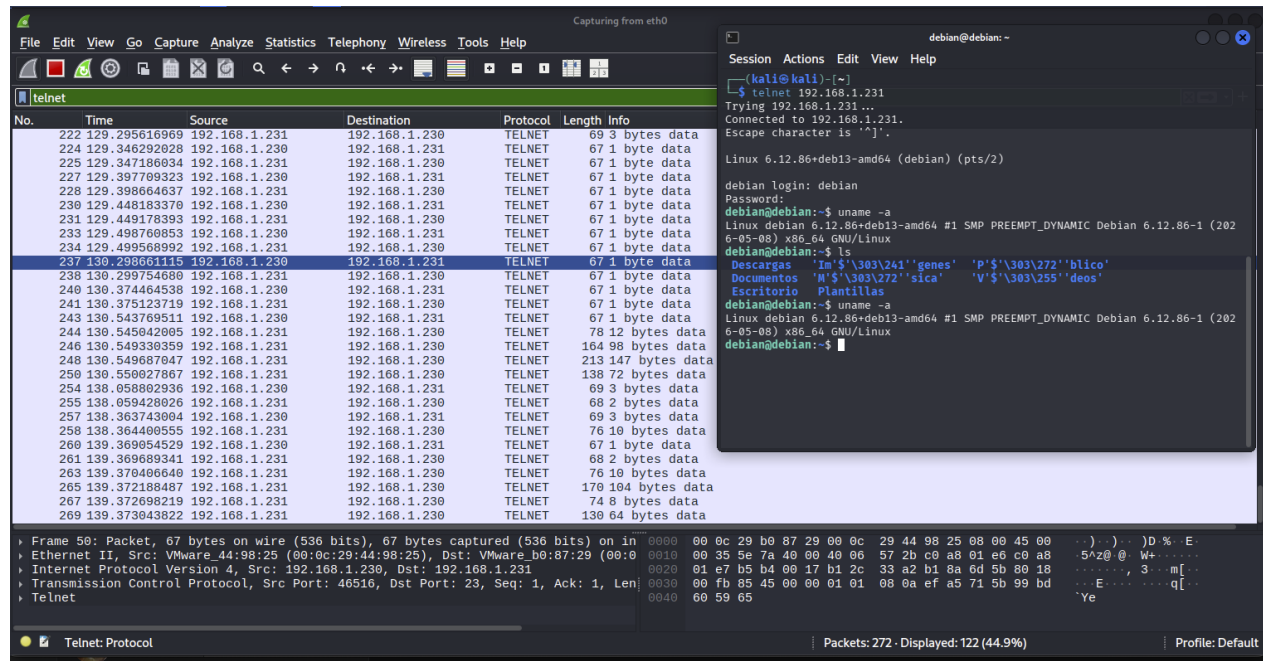


Figura 4: Captura de trafico telnet con Wireshark

Revisando uno de los paquetes capturados, vemos que el payload incluye el resultado de `uname -a` lo cual es correcto por lo que se ejecuta desde la terminal.

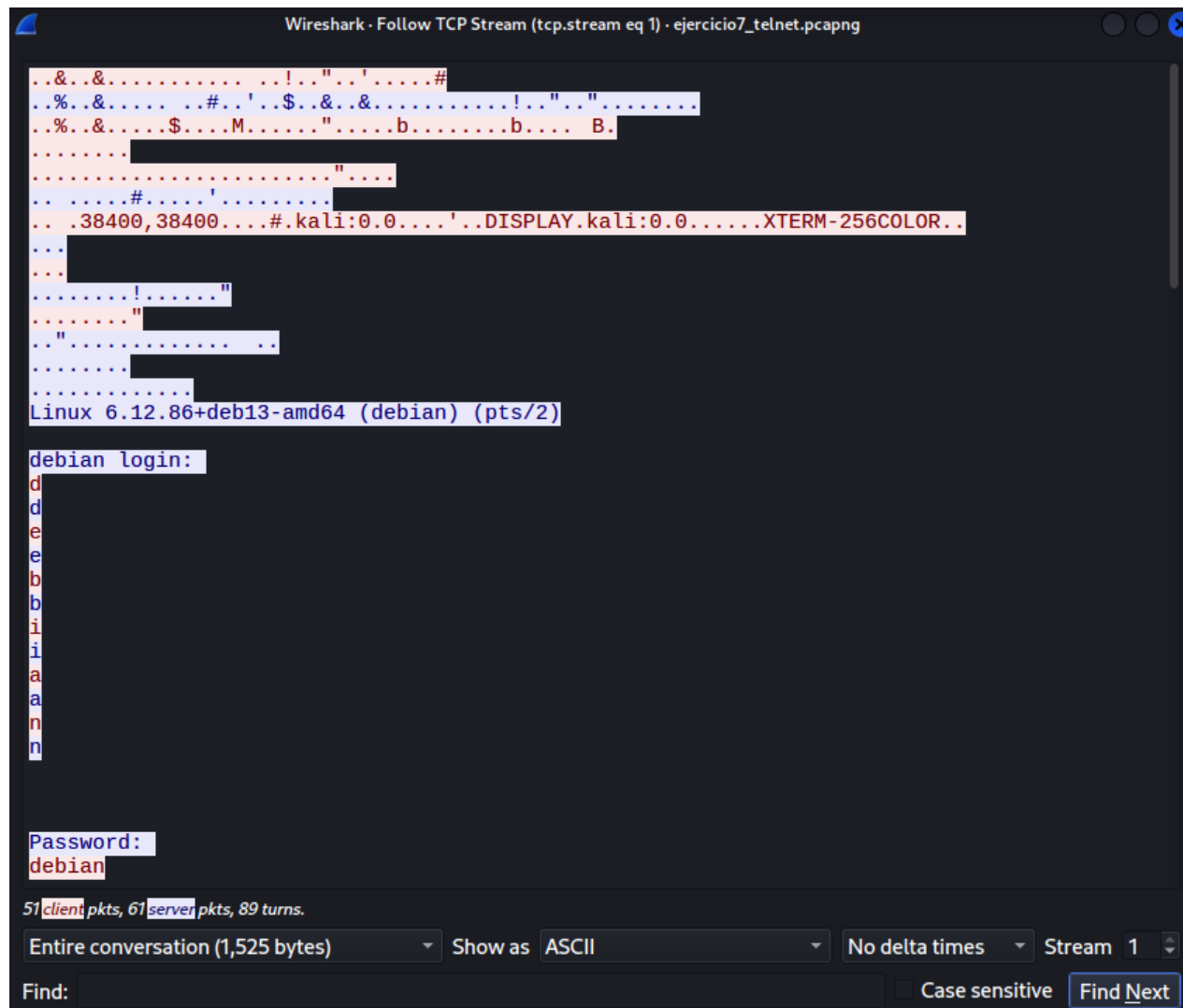


Figura 5: Revisión del payload de un paquete telnet con Wireshark

Luego, haciendo la conexión por ssh desde el host a la VM Debian usando `ssh debian@192.168.1.231` y aplicando el filtro de tráfico ssh en Wireshark, se obtiene lo siguiente:

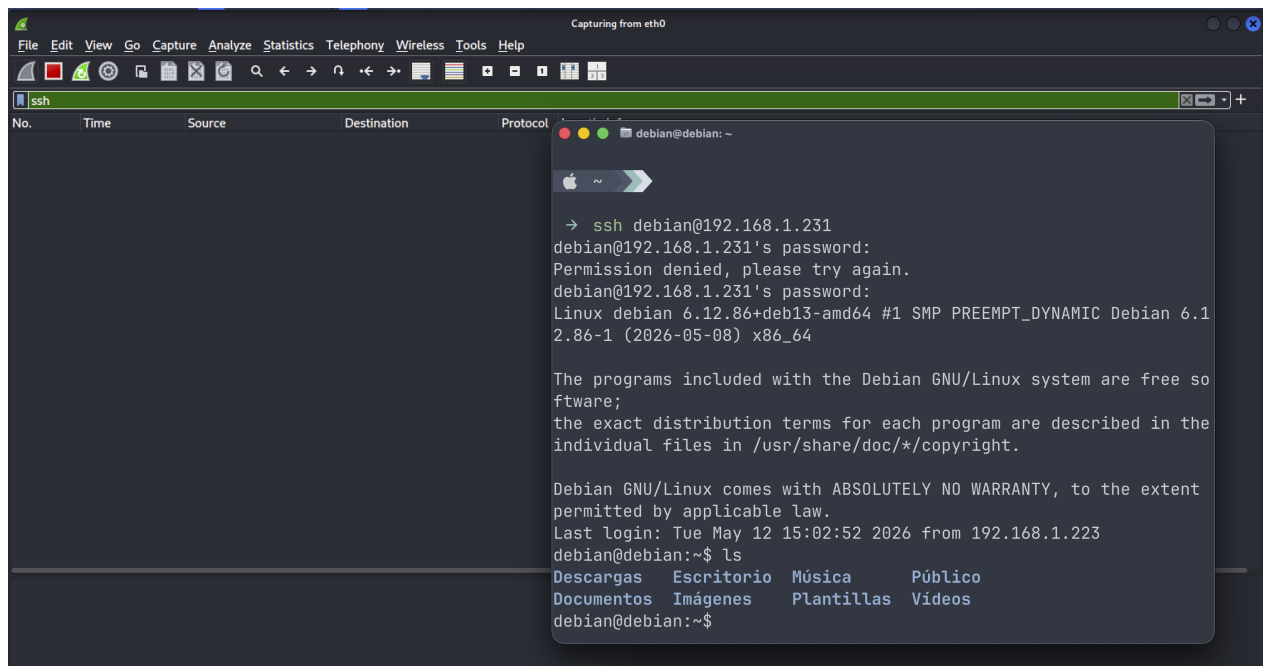


Figura 6: Captura de tráfico ssh con Wireshark

Aunque en primera instancia no se capturan paquetes de esta conexión, esto es de hecho el resultado esperado ya que el tráfico ssh está cifrado, por lo que no se puede obtener información útil de este tráfico.

Esto concuerda con las diferencias de seguridad entre **telnet**, que transmite la información en texto plano, y **ssh**, que cifra la información transmitida.